

CSIS Data Science Master Cheat Sheet

Compiled directly from CSIS3754 & CSIS3764 Past Papers (2022-2025)

1. Data Import & Inspection

Loading & Basic Inspection

```
import pandas as pd
import numpy as np

# Load Data
df = pd.read_csv('data.csv')

# Shape, Dimensions & Info
print(f"Rows: {df.shape[0]}, Cols: {df.shape[1]}")
print(f"Total elements: {df.size}")
df.info() # Memory, datatypes, non-nulls
display(df.head(20))
display(df.tail(20))
```

Summary & Value Counts

```
# Statistical Summary
display(df.describe(include='all'))

# Count unique values per feature
print(df.nunique())

# Percentage of specific condition
perc = (len(df[df['Col'] == 'Value']) / len(df)) * 100
print(f"{perc:.2f}%")
```

2. Data Cleaning & Manipulation

Dropping & Renaming

```
# Rename columns
df = df.rename(columns={'Old': 'New'})

# Drop specific columns
df = df.drop(columns=['Col1', 'Col2'])

# Drop first and last 7 columns without names
df = df.iloc[:, 1:-7]

# Shuffle and reset index
df = df.sample(frac=1, random_state=42)
df = df.reset_index(drop=True)
```

Missing Values & Zeroes

```
# Check missing percentages
print((df.isnull().sum() / len(df)) * 100)

# Discard rows with ANY missing values
df = df.dropna()

# Augment (Fill) missing with mean of column
df['Col'] = df['Col'].fillna(df['Col'].mean())

# Check for zero values
print((df == 0).sum())
```

3. Exploratory Data Analysis (EDA) & Visualization (Seaborn/Matplotlib)

Correlation Matrix & Heatmap

```
import seaborn as sns
import matplotlib.pyplot as plt

corr = df.select_dtypes(include='number').corr()
plt.figure(figsize=(10, 8))
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.show()
```

Pie & Bar Charts

```
# Pie Chart (with percentages rounded)
counts = df['Category'].value_counts()
plt.pie(counts, labels=counts.index,
autopct='%.1f%%')
plt.show()

# Overlaid Bar Charts (Red vs Blue)
plt.bar(df['Year'], df['Total_Fatals'],
color='red')
plt.bar(df['Year'], df['Aboard_Fatals'],
color='blue')
plt.show()
```

Scatter & Category Plots

```
# Single Scatter Plot
sns.scatterplot(data=df, x='Col1', y='Col2')

# Combined Scatter with Hue (3 variables)
sns.scatterplot(data=df, x='Income',
y='Credit',
hue='Card_Category')

# Category Plot
sns.catplot(data=df, x='Education',
hue='Card_Category', kind='count')
plt.show()
```

4. Pre-Processing: Encoding, Scaling & Balancing

Categorical Encoding (Text to Numeric)

```
# 1. Binary Columns (2 unique values) -> [0, 1]
bin_cols = [c for c in df.columns if
df[c].nunique() == 2]
for col in bin_cols:
    df[col] =
df[col].astype('category').cat.codes

# 2. Multi-Class (Vector Space / One-Hot
Encoding)
# Note: Exams often ask to use 0, 1, 2... for
some cols
# OR One-Hot Encoding for others.

# Label Encoding (0, 1, 2, n)
df['Col'] =
df['Col'].astype('category').cat.codes

# One-Hot Encoding (Vector Space)
df = pd.get_dummies(df, columns=['VectorCol'])
```

Scaling & Balancing

```
# Standard Scaling (Same scale)
from sklearn.preprocessing import
StandardScaler
scaler = StandardScaler()
cols_to_scale =
df.drop(columns=['Target']).columns
df[cols_to_scale] =
scaler.fit_transform(df[cols_to_scale])

# Undersampling (Balancing target)
from imblearn.under_sampling import
RandomUnderSampler
rus = RandomUnderSampler(random_state=42)
X_res, y_res = rus.fit_resample(X, y)

# Oversampling
from imblearn.over_sampling import
RandomOverSampler
ros = RandomOverSampler(random_state=42)
X_res, y_res = ros.fit_resample(X, y)
```

5. Supervised Learning: Classification (Question 2)

```
from sklearn.model_selection import train_test_split, cross_validate, learning_curve
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier

# 1. Split Data (80/20)
X = df.drop(columns=['Target'])
y = df['Target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. K-Fold Cross Validation (k=5 or k=10)
models = {'SVM': SVC(gamma='auto'), 'KNN': KNeighborsClassifier(), 'LR': LogisticRegression()}
for name, model in models.items():
    scores = cross_validate(model, X_train, y_train, cv=5, scoring=['accuracy', 'f1_macro'])
    print(f"{name} CV Accuracy: {scores['test_accuracy'].mean():.4f}")
    print(f"{name} CV F1: {scores['test_f1_macro'].mean():.4f}")

# 3. Learning Curves Plotting
train_sizes, train_scores, test_scores = learning_curve(SVC(gamma='auto'), X_train, y_train, cv=5)
plt.plot(train_sizes, np.mean(train_scores, axis=1), label='Training score')
plt.plot(train_sizes, np.mean(test_scores, axis=1), label='Cross-validation score')
plt.legend(); plt.show()

# 4. Final Evaluation on Test Set
best_model = SVC(gamma='auto').fit(X_train, y_train)
y_pred = best_model.predict(X_test)
print(accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True); plt.show()
```

6. Unsupervised Learning: K-Means & PCA (Question 3)

K-Means & Finding Optimal K

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Silhouette Score & Elbow Curve
wcss = []
sil_scores = []
K_range = range(2, 10)
for k in K_range:
    km = KMeans(n_clusters=k,
                random_state=42).fit(X)
    wcss.append(km.inertia_)
    sil_scores.append(silhouette_score(X,
                                       km.labels_))

# Plot WCSS (Elbow) and Silhouette here...

# Train Final K-Means
final_km = KMeans(n_clusters=3).fit(X)
df['Cluster'] = final_km.labels_
print("Cluster Centers:",
      final_km.cluster_centers_)
```

Principal Component Analysis (PCA)

```
from sklearn.decomposition import PCA

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

print(f"Original dim: {X.shape}, PCA dim: {X_pca.shape}")
print("Principal Components:", pca.components_)
print("Explained Variance Ratio:",
      pca.explained_variance_ratio_)

# Scatterplot of PCA components
pca_df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
pca_df['Cluster'] = final_km.labels_
sns.scatterplot(data=pca_df, x='PC1', y='PC2',
                hue='Cluster',
                palette='viridis')
plt.show()
```

Exam Theory Answers (Always Asked):

Overfitting vs Underfitting: Overfitting = high training score but low cross-validation score (large gap). Underfitting = both training and validation scores are very low.

PCA Effectiveness: "PCA successfully reduced the dimensions from N to 2, capturing X% of the variance, allowing clear 2D visualization of the clusters."

Precision/Recall/F1: Precision = Of all predicted positive, how many were true. Recall = Of all actual positive, how many did we find. F1 = The harmonic mean/balance of both.